Learning Emergent Behavior in Robot Swarms with NEAT

Pranav Rajbhandari
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA
prajbhan@andrew.cmu.edu
ORCID: 0009-0004-4933-5204

Donald Sofge
Head, Distributed Autonomous Systems Section
U.S. Naval Research Laboratory
Washington, DC
donald.sofge@nrl.navy.mil
ORCID: 0000-0003-0153-3581

arXiv:2309.14663v1 [cs.AI] 26 Sep 2023

***Abstract*—When researching robot swarms, many studies observe complex group behavior emerging from the individual agents' simple local actions. However, the task of learning an individual policy to produce a desired emergent behavior remains a challenging and largely unsolved problem. We present a method of training distributed robotic swarm algorithms to produce emergent behavior. Inspired by the biological evolution of emergent behavior in animals, we use an evolutionary algorithm to train a 'population' of individual behaviors to approximate a desired group behavior. We perform experiments using simulations of the Georgia Tech Miniature Autonomous Blimps (GT-MABs) aerial robotics platforms conducted in the CoppeliaSim simulator. Additionally, we test on simulations of Anki Vector robots to display our algorithm's effectiveness on various modes of actuation. We evaluate our algorithm on various tasks where a somewhat complex group behavior is required for success. These tasks include an Area Coverage task, a Surround Target task, and a Wall Climb task. We compare behaviors evolved using our algorithm against *designed policies*, which we create in order to exhibit the emergent behaviors we desire.**

***Index Terms*—Swarm robotics, Emergent behavior, Evolutionary algorithm, Neuroevolution**

I. Introduction

A. Emergent Behavior

Emergent behavior is a phenomenon observed in swarms of agents. It is generally defined as a complex swarm behavior which occurs as a consequence of each individual agent following a relatively simple control scheme [13], [15]. Examples of this can be found in the behavior of groups of animals, such as how some species of fire ants have been observed to create rafts out of their bodies to survive flooding [11]. In this case, the individual behavior of 'grab onto neighbors' creates the emergent behavior of 'build a raft'. Emergent behavior is also exhibited in migrating swarms of some species of caterpillars, which walk on top of each other in order to create a 'rolling swarm' [19]. This allows the caterpillars to migrate quicker than simply walking. Similar instances of complex emergent behavior have been observed in the field of swarm robotics.

B. Swarm Robotics

Though there are various definitions of swarm robotics, it is generally agreed that robot swarms are multi-agent systems where each agent is autonomous and has low complexity [3], [8], [12]. They are used for various tasks, including exploration and surveillance. They are characterized by using locally communicating distributed systems as opposed to a central controller. Due to this, they remain functional as the swarm size is increased. However, they do not have a central controller, so careful fine tuning of the individual policies is required to achieve a desired swarm behavior.

It is well known that emergent behavior can arise in robot swarms, and past studies have explored this. Pagello et al. [16] (2001) study how to make a robot swarm perform a cooperative task by creating emergent behaviors. They implement this by dynamically assigning predefined roles to each robot. A function Q is learned for each agent, which takes in local information and outputs the best role for the agent to adopt. After refining their Q functions, they observe cooperative emergent behavior in their chosen task, robot soccer.

In a more recent study, Oliveri et al. [14] use a Monte Carlo scheme to continuously refine the behavior of individual agents in a swarm. The robots they choose to use are arranged in a line, and connected with soft tubes. Their actuation is in the form of a pump, which increases the distance between a robot and the neighbor behind it. In training, each robot attempts to learn its optimal phase time (the time that it takes for a cycle of the pump filling then unfilling the tube). The robots randomly sample nearby phase times, and update depending on the change in velocity they experience. The study concludes that the result of each agent attempting to optimize its velocity is an emergent behavior where the group of connected robots crawls forward.

Similar to the research of Oliveri and Pagello, we focus on optimizing an individual policy to best create a desired emergent swarm behavior. Inspired by how biological evolution was effective in creating various emergent behaviors in animals, we propose using an evolutionary algorithm to solve this task.

C. Evolutionary Algorithms/NEAT

Evolutionary algorithms are algorithms inspired by biological evolution, where a fitness function is optimized by repeatedly mutating and evaluating members of a 'population' of solutions. Importantly, these methods are generic, as they do

DISTRIBUTION STATEMENT A: Approved for public release. Distribution is unlimited.

not make any assumptions about the format of the solutions or the topography of the fitness function. Since we want to solve a problem with general inputs/outputs, we use neuroevolution (NE), a type of evolutionary algorithm that evolves neural networks. In early NE algorithms, the topology[1] of the neural network is initially set, and the mutations take place in the weights of the connections [22]. However, using the Neuroevolution of Augmenting Topologies (NEAT) algorithm, we are able to evolve the structure of the network as well [23]. This is useful since it allows us to begin with a simple neural network that evolves in complexity according to the requirements of the task.

Thus, we evaluate NEAT as a candidate for evolving emergent behavior in robot swarms. The method we propose is using a homogeneous swarm (where each agent is using the same neural network) to act in an experiment. At the end of the experiment, the 'performance' of the swarm is evaluated with some user-defined metric, and we use this as the fitness function for the neural network. This allows us to use NEAT to evolve a population of neural networks to better exhibit a desired emergent behavior. A trade off with this is that after running a simulation, we are left with a single fitness score for the network, in contrast to the many times we actually call the network.

However, this seems to be an issue inherent to our problem, since the performance of a swarm can only be evaluated when considering the behavior of the swarm as a whole. Thus, the actions taken by each individual agent in a swarm are difficult to assign a reward to. This makes alternate learning algorithms like Reinforcement Learning (RL) difficult to use, since they require rewards to be assigned to every action taken. In contrast, this is coherent with the evolutionary structure we use in our algorithm, since each 'evaluation' only needs to yield one fitness value.

D. Organization

The rest of the paper is organized as follows: Section II discusses related work. Section III details our proposed learning algorithm, as well as common sensing and output schemes we use. We present our experiments in Section IV, and discuss the results in Section V. The conclusions we draw are in Section VI.

II. Related Work

Much research has focused on the goal of learning individual behaviors for a robot swarm [2], [3], [5], [6], [14], [16]. Behjat et al. [2] explore how to learn tactical swarm behavior through a combination of various techniques. These techniques include learning neural network based robot policies, dynamically organizing the swarm into groups, and Pareto filtering of points of interest to reduce the problem dimensionality.

Fan et al. [5] compare different swarm intelligence algorithms against each other to evaluate their relative performance in obstacle performance under different circumstances. The algorithms they evaluate are the bat algorithm (BA), particle swarm optimization (PSO), and the grey wolf optimizer (GWO). They find that PSO outperforms BA, which outperforms GWO in general. However, GWO performs better than the other two algorithms in the case of large swarms and large communication ranges.

Dorigo et al. [6] explore how to create communication protocols between agents in a swarm that best help methods such as deep reinforcement learning create good decentralized control policies .

In addition to the studies by Oliveri and Pagello which focus on the learning aspect (described in Section I-B), there has also been research on helping the stability of emergent behaviors in swarms. In 2022, Chen and Ng explore secure communications between robots in a swarm [3]. They model these communications as a series of random graphs, and use a method involving hash chains to identify 'rogue robots' with high probability. This allows them to ensure that the emergent behavior exhibited by the swarm is protected. In their paper, they also create a system to distinguish different classes of robot swarms by identifying differences in the robot homogeneity, the interactions between robots, and the interactions with a central control.

Our work is closely related to research done by Trianni et al. in 2003 [26]. They use an evolutionary algorithm to evolve an 'aggregation' behavior in a swarm of robots, inspired by the self-organized aggregation behavior observed in the cellular slime mold *Dictyostelium discoideum* [27]. Trianni's evolutionary algorithm uses a neural network with no hidden layers, and evolves by mutating the weights. They use a fitness function that seeks to minimize the mean distance of each agent from the swarm's center of mass. Later research by Bahceci explores this further by varying the parameters of the evolutionary algorithm to see how it affects the evolution of aggregation behaviors [1].

We expand on Trianni's idea by generalizing the algorithm to allow user-defined fitness functions. Additionally, we use the NEAT algorithm as opposed to simply evolving the weights of a set network. These extensions allow our algorithm to theoretically evolve a network to arbitrary complexity in order to create a desired emergent behavior.

We use the GT-MABs as the robots for our experiments [25]. The control system for their actuation was designed based on the dynamic model created in [4], [24].

III. Methodology

A. Evolutionary Algorithm

In Algorithm 1, we apply the NEAT algorithm to evolving a swarm of robots to exhibit some desired emergent behavior. The experiment setup E includes the objects/agents involved, as well as the definition and implementation of the network inputs and outputs. In our case, we define E through a CoppeliaSim simulation [18]. The evaluation function F evaluates the overall behavior of the swarm throughout the experiment and returns a real number representing the 'fitness' of the swarm. F can either directly evaluate the desired emergent

[1]The topology of a neural network refers to the graph structure of the nodes and the connections between them

Algorithm 1 Swarm Evolutionary Algorithm

Input E: Experiment setup
F: Swarm evaluation function

$G \leftarrow$ Population of randomly initialized neural networks
loop For desired # of generations
  for network g in G **do**
    Initialize robot swarm experiment ▷ Using E
    Give each robot g for control
    Run experiment to termination
    $f_g \leftarrow$ swarm fitness ▷ evaluated with F
  end for
  $G' \leftarrow$ NEAT reproduction on G ▷ Fitness $f_g : g \in G$
  $G \leftarrow G'$ ▷ Update next generation
end loop ▷ Note: parallelization is possible since each genome evaluation is independent

behavior, or evaluate some aspect of the swarm correlated with it. To implement this, we use the NEAT-Python package [10].

B. Network Inputs

The inputs into the neural network vary for each task, but usually include the following method of sensing neighbors.

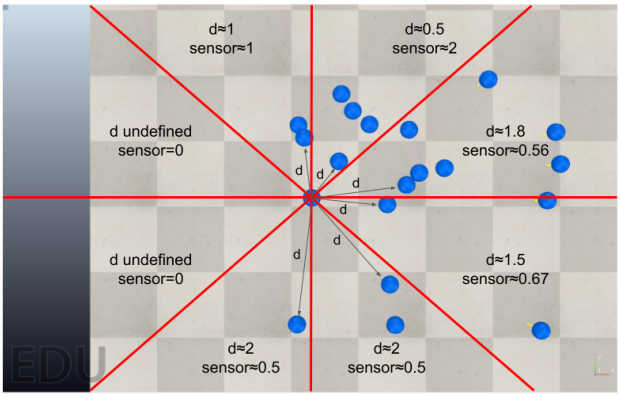


Fig. 1: k-tant distance sensing with $k = 8$

We consider the polar coordinates of each neighbor and split the angular (θ) parameter into k regions. We allow each agent to sense the inverse distance to the nearest robot on each k-tant, as shown in Fig. 1. We use the inverse distance since having the sensor read 0 when not detecting neighbors made more sense with this mapping. It is continuous in the sense that the limit of the sensor output as the closest neighbor gets arbitrarily far away is the same as the value when not sensing a neighbor.

We also experiment with allowing each agent to sense the number of neighbors within a range on each k-tant instead, as shown in Fig. 2. We introduce this variant in certain tasks to explore if it is more helpful for an agent to know the proximity or the count of its neighbors in a given direction.

For 3D cases, we can extend this sensing method into 'ℓ-k-tant' sensing. We consider the spherical coordinates of each neighbor and split the ϕ and θ parameters into ℓ parts and k parts respectively, for a total input dimension of $\ell \cdot k$. However, for our experiments we only use k-tant sensing.

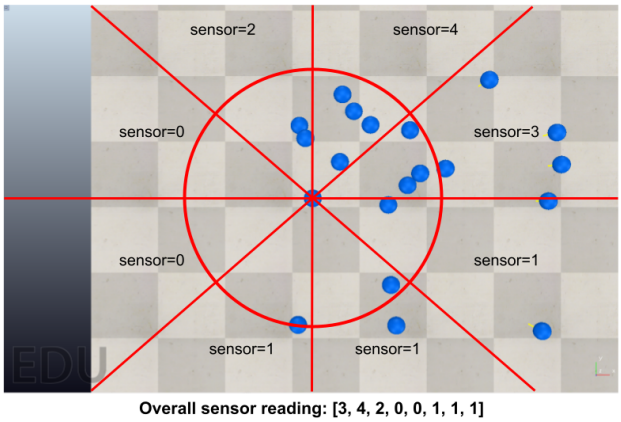


Fig. 2: k-tant neighbor sensing with $k = 8$

C. Network Outputs/Agent Actuation

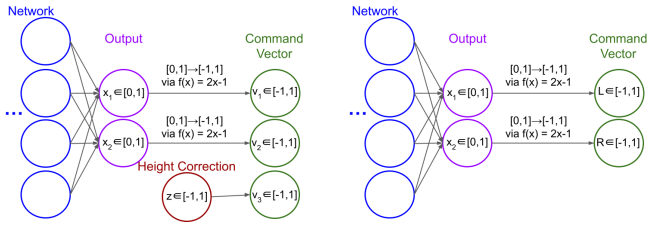


Fig. 3: GT-MAB (left) and Anki Vector (right) actuation scheme

To actuate the GT-MABs, we use a velocity controller that takes input in $\mathbb{R}^3$. Thus, we must transform the output of the neural network into a velocity vector, as shown in Fig. 3. For our experiments, we want the GT-MABs to keep a certain height, so we directly code the z component to keep the blimp at 1 meter. We set the output dimension of the evolved neural networks to 2, and assign one output to the x component and one to the y component. Additionally, since the neural network uses a sigmoid activation, the output is on the interval $[0, 1]$. We transform this to allow a command vector on $[-1, 1]^3$.

We also run experiments using a simulation of the Anki Vector robots, which use a wheel velocity controller. We believe this is a significant variation from a global velocity controller since the motion of the agent is dependent on the heading angle as well as the network output. To go in a particular direction, an agent must rotate to face that direction before moving forward. Thus, we use the Anki Vector experiments to evaluate if our algorithm is robust to variations in the actuation scheme.

As shown in Fig. 3, we use similar transformations to the GT-MABs to result in a command vector in $[-1, 1]^2$. We use this to actuate the Anki Vectors by assigning one dimension

to the left wheel velocity and the other to the right wheel velocity.

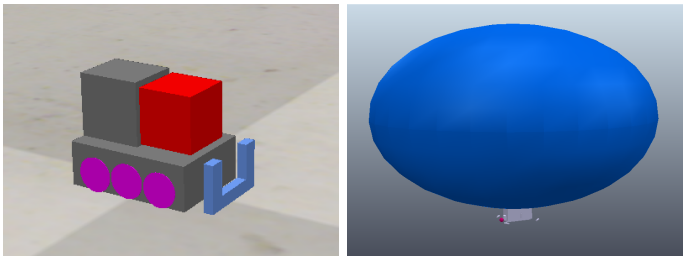


Fig. 4: CoppeliaSim model of Anki Vector and GT-MAB

IV. Experiments

To test our method, we create various tasks for the robot swarms to complete. We aim to make the tasks achievable through a swarm behavior. Since each agent's neural network is only given local information, the algorithm must evolve an individual policy that produces emergent behavior for the swarm to perform well.

For each experiment, we train our evolutionary algorithm for 50 generations in a CoppeliaSim physics simulation. The agents we use are simulated versions of the GT-MAB and the Anki Vector robots (Fig. 4). We compare the performance of the evolved policy in each experiment to the performance of a *designed policy* that we hard-code to solve the task.

We use ROS2 Foxy to handle transferring sensing and actuation messages between CoppeliaSim and Python [9]. Our full implementation is available on Github [17].

A. Task: Area Coverage

In this task, we simulate a 'search and rescue' scenario. The environment we use is an empty square arena with the agents spawned randomly near the center (Fig. 6 & Fig. 7). For the fitness function, we adapt deployment entropy, a measure of how well distributed the agents become in the environment [7]. Deployment entropy is defined by discretizing the environment into a grid, and measuring the entropy of the distribution of agents in that grid, as shown in Fig. 5. Since entropy is maximized by a uniform distribution, we believe using this as a fitness function would encourage the agents to spread out, which is the desired behavior for a 'search and rescue' mission. We implement this task for the both the Anki Vector robots and the GT-MABs.

1) GT-MABs: For the GT-MABs, we use 20 agents and divide the environment into a grid with 16 units for calculating deployment entropy. We terminate each experiment after 60 seconds. We use a variation of Octant Distance Sensing[2] shown in Fig. 1 as the input. We additionally allow the agents the ability to sense the border walls, implemented by counting the closest wall point on each octant as an additional agent. We add this capability since the agents would otherwise have

[2] k-tant distance sensing with $k = 8$

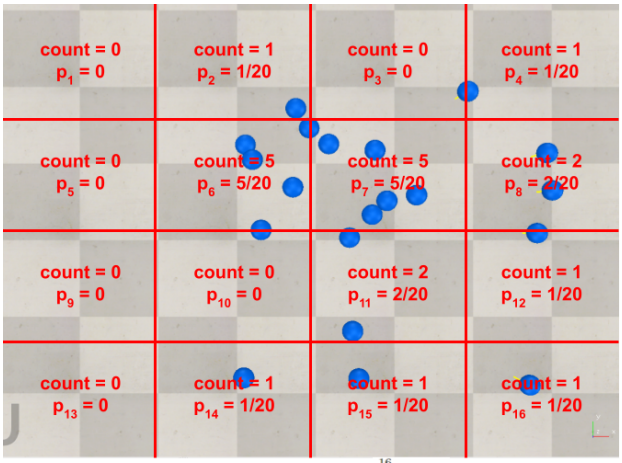


Fig. 5: Calculation of deployment entropy

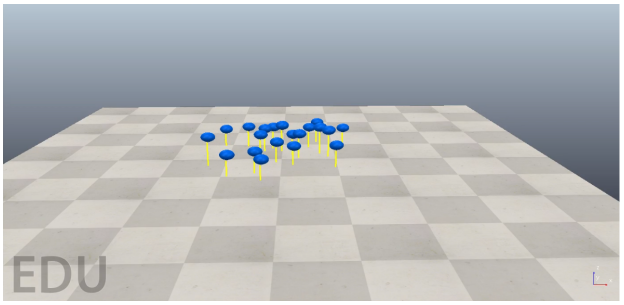


Fig. 6: GT-MAB Area Coverage experiment initial position

no way of sensing the boundaries. For output, we use our GT-MAB actuation scheme (Fig. 3).

We define a *designed policy* where each agent moves away from the closest neighbors. We expect this to cause the agents to distribute themselves evenly, producing the desired swarm behavior.

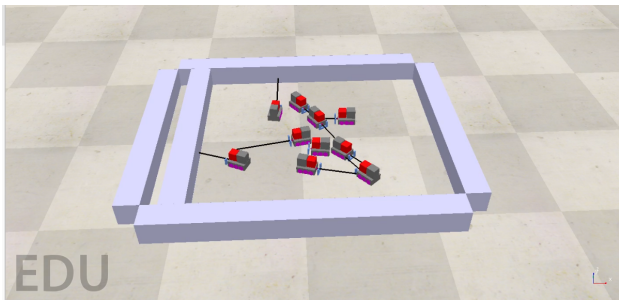


Fig. 7: Anki Area Coverage experiment initial position

2) Anki Vectors: For the Anki Vectors, we use 10 agents and divide the environment into a grid with 9 units for calculating deployment entropy. As before, we terminate each experiment after 60 seconds.

We use Octant Distance Sensing (Fig. 1) along with the Anki's proximity sensor as the input. In this case, since

the proximity sensor is able to sense the boundary walls, we do not allow the agent to sense the border with Octant Distance Sensing. For output, we use the Anki's wheel velocity actuation scheme (Fig. 3).

We implement a similar *designed policy* for each Anki Vector to move away from its closest neighbors. We notice that this policy is more difficult to design, since we must include logic to turn the agent to face a more desirable direction. This supports our assumption that the control scheme of the Anki Vector robots is different enough from the GT-MAB's to evaluate our algorithm's robustness to control schemes.

B. Task: Wall Climb

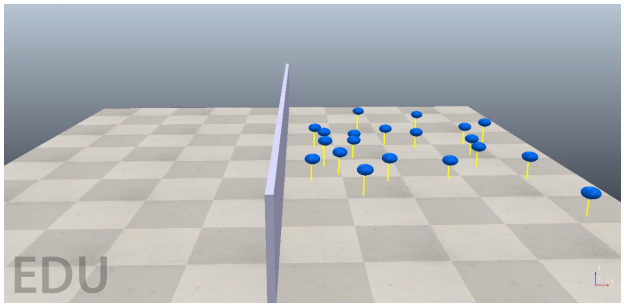


Fig. 8: Wall Climb experiment initial position

In previous research we established that a swarm of GT-MABs was able to climb a wall that was much taller than the altitude they were assigned to hold [20]. Upon closer inspection, we realized that that collisions were resulting in the agents stacking on top of one another. Since they read their altitude using ultrasonic sensors, the top GT-MAB would register the other one as a floor, effectively adding their desired heights. A stack of three agents would allow the top one to pass over the wall.

We also note that the 'wall climbing' behavior is susceptible to the sensing angle of the GT-MAB's ultrasound-based range sensor. With a large sensing angle, the GT-MABs are able to sense the wall itself when they are next to it, which results in a single agent being able to climb the wall on its own. To avoid this, we use a narrow ultrasound angle so that the agents must stack in order to climb the wall.

1) Octant Distance Sensing: We design the Wall Climb task to replicate this scenario. The environment we use is an arena with a 3 meter tall wall on the y axis. The GT-MABs spawn randomly on the right side, as shown in Fig. 8. For the fitness function, we use the number of agents that end the experiment on the other side of the wall. If no agents succeed, we subtract a penalty of the distance of the closest blimp to the wall. We believe using this as a fitness function will encourage the "aggregation" behavior, since this is the only way the agents can climb the wall. We use 20 agents, and terminate each experiment after 60 seconds.

We use Octant Distance Sensing as our input (Fig. 1) and our GT-MAB actuation scheme as output (Fig. 3).

We define a *designed policy* where each agent moves towards their closest neighbor, as well as slightly towards the direction of the wall. We expect this to cause the GT-MABs to flock and stack on top of one another. Once they are stacked, we predict they will slowly make their way over the wall, producing the desired swarm behavior.

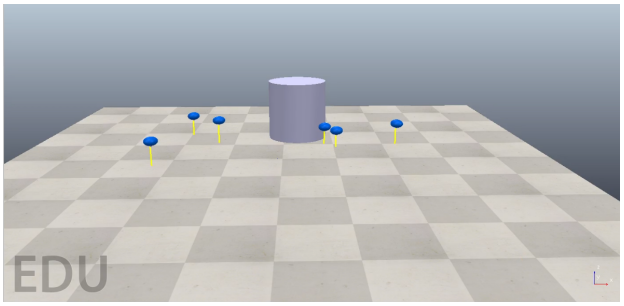


Fig. 9: Surround Target experiment initial position

2) Octant Neighbor Sensing: We also compare this with using the Octant Neighbor Sensing scheme as our input, as shown in Fig. 2. Our *designed policy* for this scheme is similar, where we move towards the direction with the highest neighbor count. We predict that sensing the number of neighbors is more useful to create the 'flocking' behavior we desire. Thus, we expect that this sensing scheme will be more successful.

C. Task: Surround Target

In this task, we simulate a goal of securing a perimeter around a target [21]. The environment we use is an arena containing a cylinder target spawned randomly, with the GT-MABs also spawned randomly in the arena.

For the fitness function, we consider all the agents close to the target. Using polar coordinates centered at the target, we find the measure of the largest uncovered arc a. We set the fitness to $-a$ since we want to minimize this value. We also subtract a penalty of the distance of the closest agent to the target for experiments where no agents are near the target. We use 6 agents and terminate each experiment after 60 seconds.

We include Octant Distance Sensing in our input (Fig. 1), as well as the agent's displacement vector from the target. For output, we use our GT-MAB actuation scheme (Fig. 3).

We define a *designed policy* where each agent moves towards the target, as well as slightly away from their closest neighbor. We expect this to cause the GT-MABs to head for the target, then slowly spread out, producing the desired swarm behavior of surrounding the target.

V. Results

For each experiment, after training for 50 generations, we compare the fitness of the best genome in the final generation with the fitness of our *designed policy*. We run each for 60 trials in their respective experiments and collect the fitness mean and standard deviation. We report our results in Table I.

TABLE I: Comparison of Fitness in Evolved[a] and Designed Behaviors

Experiment	Behavior	Mean	Stdev.
GT-MAB Area Coverage	Evolved	2.53	0.10
	Designed	2.62	0.12
Anki Area Coverage	Evolved	2.09	0.083
	Designed	1.87	0.20
Wall Climb (Distance Sense)	Evolved	16.70	1.12
	Designed	12.13	3.78
Wall Climb (Neighbor Sense)	Evolved	16.72	1.32
	Designed	15.33	1.58
Surround Target	Evolved	-2.21	.70
	Designed	-1.24	0.14

[a]: Using the best genome from the final generation.

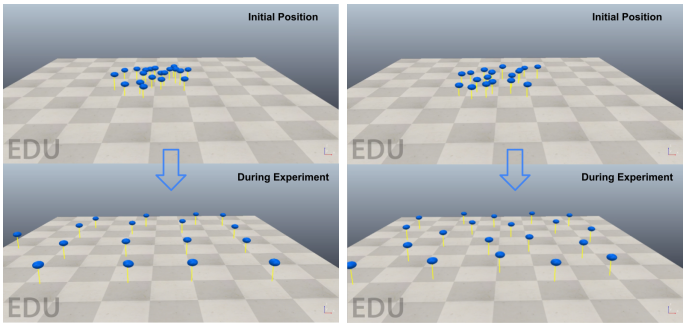


Fig. 10: GT-MAB Area Coverage evolved behavior (left) and *designed policy* behavior (right)

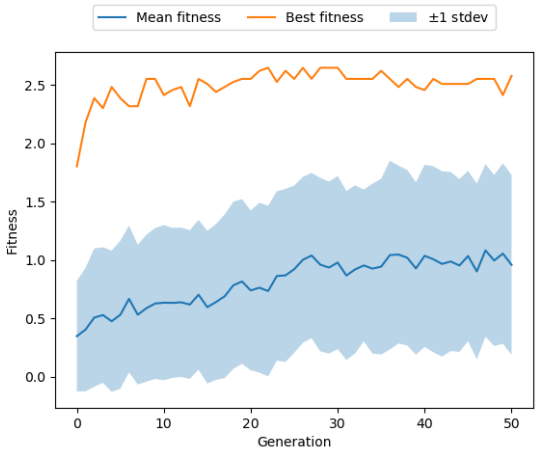


Fig. 11: GT-MAB Area Coverage population fitness across generations

A. Area Coverage

1) GT-MABs: After training the GT-MAB models in CoppeliaSim, we observe that the behavior they evolved seems to be for each agent to move in a direction away from its close neighbors. Fig. 10 shows an experiment run with the network with the best genome in the last generation of training. The result of this behavior seems to be a well distributed swarm, with the agents arranged very close to a lattice. From Fig. 11, we can see that after around the 15th generation, the best genome of each generation achieved close to the theoretic maximal entropy[3].

The result of the *designed policy*, shown in Fig. 10, also had the swarm distribute in the environment, forming a similar lattice. Comparing these results, we can see that the *designed policy* outperforms the evolved behavior by about one standard deviation (Table I).

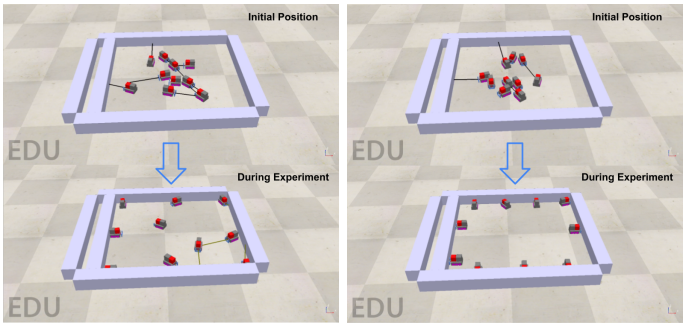


Fig. 12: Anki Area Coverage evolved behavior (left) and *designed policy* behavior (right)

Overall, we show that in this task, our algorithm learns a local behavior that closely approximated the desired 'search and rescue' emergent swarm behavior. From the slight upwards trend in the mean of Fig. 11, we expect that the evolved results may perform closer to the *designed policy* with more training.

2) Anki Vectors: After training the Anki Vector models in CoppeliaSim, we observe the behavior appears to be for each agent to spin in place, and move away from close neighbors directly in front of or behind it. Fig. 12 shows an experiment run with the best genome in the last generation of training. The result of this behavior appears to be a well distributed swarm. From Fig. 13, we can see that after a few generations, the best genome of each generation achieved the theoretic maximal entropy[4].

The result of the *designed policy*, shown in Fig. 12, had the swarm distribute reasonably well in the environment. Comparing these results, we can see that the evolved behavior outperforms the *designed policy*, since the mean is about one standard deviation higher (Table I). This seems to be due to the *designed policy* favoring sending agents to the edges of the environment.

[3]2.718 from placing one agent in 12 of the grid units, and two agents in 4 of the units

[4]2.16 from placing one agent in 8 of the grid units, and two agents in 1 of the units

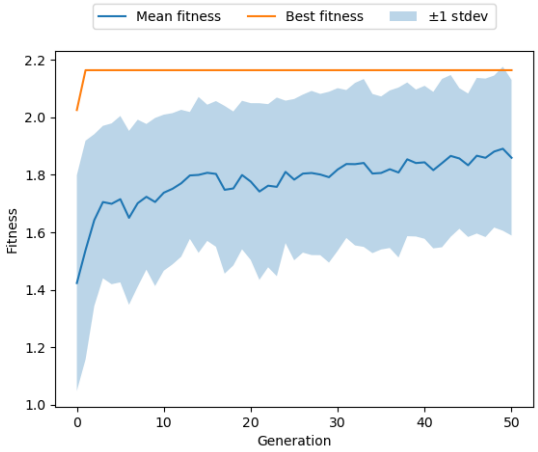


Fig. 13: Anki Area Coverage population fitness across generations

Overall, we show that in this task, our algorithm learns a local behavior that closely approximated the desired 'search and rescue' emergent swarm behavior. The evolved behavior greatly outperforms the *designed policy*, which shows that our evolutionary algorithm performs comparably to designing the behavior with knowledge of the task.

B. Wall Climb

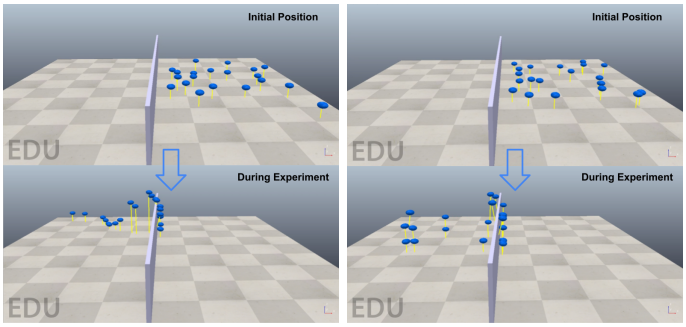


Fig. 14: Distance Sensing Wall Climb evolved behavior (left) and *designed policy* behavior (right)

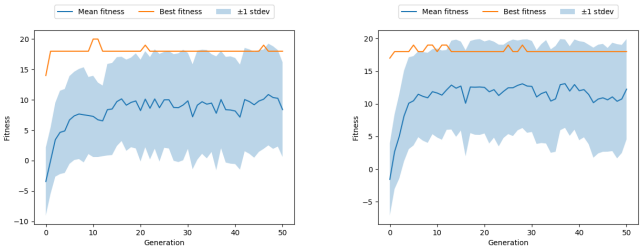


Fig. 15: Distance Sensing Wall Climb (left) and Neighbor Sensing Wall Climb (right) population fitness across generations

1) Octant Distance Sensing: After training the GT-MAB models in CoppeliaSim, we observe that the behavior they evolved seems to be for each agent to move in a direction towards its closest neighbors, in addition to moving in the direction to climb the wall. Fig. 14 shows an experiment run with the network with the best genome in the last generation of training. The result of this behavior does seem to be a flock of agents stacked on top of each other, climbing the wall. From Fig. 15, we can see that after a few generations, the best genome of each generation achieved having about 18 GT-MABs make it over the wall.

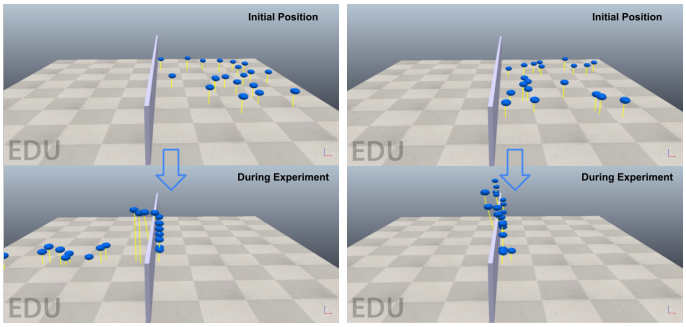


Fig. 16: Neighbor Sensing Wall Climb evolved behavior (left) and *designed policy* behavior (right)

2) Octant Neighbor Sensing: As for the Neighbor Sensing experiment, we noticed both sensing methods achieved similar population fitness values across generations (Fig. 15). The flocking behavior evolved was also similar in both experiments (Fig. 14, Fig. 16). From Table I, we see there is no statistically significant difference between the two fitness results[5].

The result of the *designed policy* in both of these experiments similarly causes the agents to flock together and climb the wall. However, in this case we do notice a difference in the two modes of sensing. In the Distance Sensing experiments, the *designed policy* seems to be more likely to cause the agents to form several smaller groups as opposed to one large one. This results in a lower fitness due to more agents being left behind, as shown in Table I. Comparing this with our evolved behavior, we can see in both experiments that the evolved behavior outperforms the best *designed policy* by about one standard deviation.

Overall, we show that in this task, our algorithm learns a local behavior that closely approximated the desired 'flocking' emergent swarm behavior. We can further see that although the different sensing modes has an effect on our *designed policy*, the evolutionary algorithm is robust to these variations.

C. Surrounding a Target

After training the GT-MAB models in CoppeliaSim, we observe that the behavior they evolved seems to be for each

[5]We perform a two sample t-test with unequal variance on the fitnesses obtained from the evolved Neighbor Sensing and Distance Sensing experiments. Using their respective means and standard deviations of (16.70,1.12) and (16.72,1.32) with a sample size of 60 for both, we arrive at a p-value of $p > .9$, which is much larger than .05, the accepted value for statistical significance. This shows that with our sample size, there is no significant difference between the fitnesses obtained from the different sensing methods.

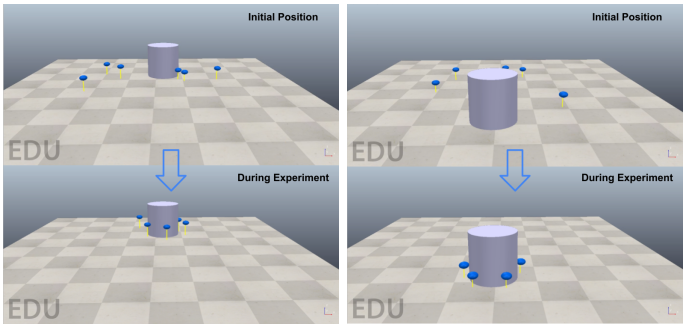


Fig. 17: Surround Target evolved behavior (left) and *designed policy* behavior (right)

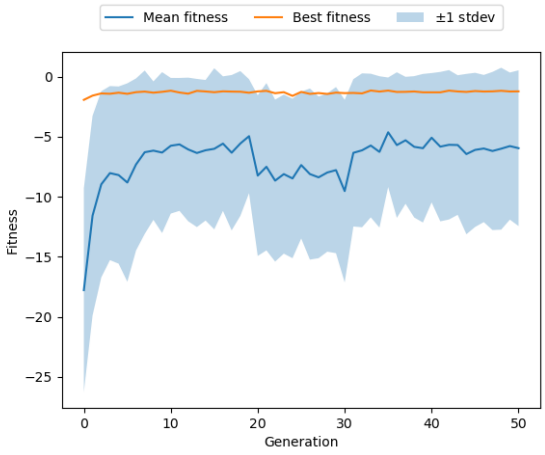


Fig. 18: Surround Target population fitness across generations

agent to spiral around the target, in addition to moving away from its closest neighbor. Fig. 17 shows an experiment run with the network with the best genome in the last generation of training. The result of this behavior does seem to be a flock of agents surrounding the target. From Fig. 18, we can see that after a few generations, the best genome of each generation achieves having a largest uncovered arc of about 2 radians.

The result of the *designed policy*, shown in Fig. 17, also caused the agents to surround the target. Comparing these results, we can see that our *designed policy* outperforms the evolved behavior by at least one standard deviation (Table I).

Overall, we show that in this task, our algorithm learns a local behavior that closely approximates the desired emergent swarm behavior. However, our *designed policy* still does outperform the evolved behavior, which suggests that for some tasks it is more difficult to evolve the optimal behavior. We believe fine tuning of the evolution parameters may help this issue, but that is outside the scope of our research.

VI. Conclusion

In this paper, we present a novel extension of the NEAT algorithm designed to learn emergent behaviors in robot swarms. The algorithm we present is able to be applied on robot swarms with various modes of sensing and actuation. Results from simulations show that individual agent behaviors evolved using this method are comparable to hand designed policies at producing desired complex emergent behaviors.

In future research, we plan to test our evolved policies on the physical GT-MABs and Anki Vector robots. We also plan to evaluate our algorithm on a more complex set of tasks. We also may explore the fine tuning of NEAT parameters to try improving our results. Additionally, the accuracy of the physics evaluation seems to be dependant on the performance of the computer, as this determines the speed that each agent can respond to stimuli. In this paper, we choose to speed up our experiments by running multiple simulations in parallel. However, using less parallelization or using a more powerful computer could improve the results of our algorithm.

References

[1] Erkin Bahceci and Erol Sahin. Evolving aggregation behaviors for swarm robotic systems: a systematic case study. In *Proceedings 2005 IEEE Swarm Intelligence Symposium, 2005. SIS 2005.*, pages 333 – 340, 07 2005.

[2] Amir Behjat, Hemanth Manjunatha, Prajit Krisshna Kumar, Apurv Jani, Leighton Collins, Payam Ghassemi, Joseph Distefano, David Doermann, Karthik Dantu, Ehsan Esfahani, and Souma Chowdhury. Learning robot swarm tactics over complex adversarial environments. In *2021 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, pages 83–91, Nov 2021.

[3] Liqun Chen and Siaw-Lynn Ng. Securing emergent behaviour in swarm robotics. *Journal of Information Security and Applications*, 64:103047, 2022.

[4] Sungjin Cho, Vivek Mishra, Qiuyang Tao, Paul Vamell, Matt King-Smith, Aneri Muni, Weston Smallwood, and Fumin Zhang. Autopilot design for a class of miniature autonomous blimps. In *2017 IEEE Conference on Control Technology and Applications (CCTA)*, pages 841–846, 2017.

[5] Jiaqi Fan, Mengqi Hu, Xianghua Chu, and Dong Yang. A comparison analysis of swarm intelligence algorithms for robot swarm learning. In *2017 Winter Simulation Conference (WSC)*, pages 3042–3053, Dec 2017.

[6] Maximilian Hüttenrauch, Adrian Šošić, and Gerhard Neumann. Local communication protocols for learning complex swarm behaviors with deep reinforcement learning. In *Swarm Intelligence*, pages 71–83, Cham, 2018. Springer International Publishing.

[7] John D. Kelly, Daniel M. Lofaro, and Donald Sofge. Persistent area coverage for swarms utilizing deployment entropy with potential fields. In *2020 17th International Conference on Ubiquitous Robots (UR)*, pages 479–486, 2020.

[8] Thalia May Laing, Siaw-Lynn Ng, Allan Tomlinson, and Keith M Martin. Security in swarm robotics. *Handbook of Research on Design, Control, and Modeling of Swarm Robotics*, page 42–66, 2016.

[9] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[10] Alan McIntyre, Matt Kallada, Cesar G. Miguel, Carolina Feher de Silva, and Marcio Lobo Netto. neat-python. https://github.com/CodeReclaimers/neat-python, 2008.

[11] Nathan J. Mlot, Craig A. Tovey, and David L. Hu. Fire ants self-assemble into waterproof rafts to survive floods. *Proceedings of the National Academy of Sciences*, 108(19):7669–7673, 2011.

[12] Iñaki Navarro and Fernando Matia. An introduction to swarm robotics. *ISRN Robotics*, 2013, 01 2013.

[13] Richard E. Neapolitan and Xia Jiang. Chapter 5 - decision analysis fundamentals. In *Probabilistic Methods for Financial and Marketing Informatics*, pages 177–228. Morgan Kaufmann, Burlington, 2007.

[14] Giorgio Oliveri, Lucas C. van Laake, Cesare Carissimo, Clara Miette, and Johannes T. B. Overvelde. Continuous learning of emergent behavior in robotic matter. *Proceedings of the National Academy of Sciences*, 118(21):e2017015118, 2021.

[15] Timothy O'Connor. Emergent Properties. In *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Winter 2021 edition, 2021.

[16] E. Pagello, A. D'Angelo, C. Ferrari, R. Polesel, R. Rosati, and A. Speranzon. Emergent behaviors of a robot team performing cooperative tasks. *Advanced Robotics*, 17(1):3–19, 2003.

[17] Pranav Rajbhandari. Swarm Coppeliasim. https://github.com/pranavraj575/swarm_coppeliasim, July 2023.

[18] E. Rohmer, S. P. N. Singh, and M. Freese. V-rep: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326, 2013.

[19] Destin Sandlin. Gregarious caterpillar locomotion - raw video for analysis. https://www.youtube.com/watch?v=YehR0wSUioY, Jun 2013.

[20] Tristan Schuler, Cameron Kabacinski, Daniel M. Lofaro, Dhawal Bhanderi, Jennifer Nguyen, and Donald Sofge. Wall climbing emergent behavior in a swarm of real-world miniature autonomous blimps. In *Proceedings of the 15th International Conference on Agents and Artificial Intelligence, ICAART 2023, Volume 1, Lisbon, Portugal, February 22-24, 2023*, pages 225–232. SCITEPRESS, 2023.

[21] Daigo Shishika and Vijay Kumar. A review of multi agent perimeter defense games. In *Decision and Game Theory for Security*, pages 472–485, Cham, 2020. Springer International Publishing.

[22] Kenneth Stanley, Jeff Clune, Joel Lehman, and Risto Miikkulainen. Designing neural networks through neuroevolution. *Nature Machine Intelligence*, 1, 01 2019.

[23] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.

[24] Qiuyang Tao, Jaeseok Cha, Mengxue Hou, and Fumin Zhang. Parameter identification of blimp dynamics through swinging motion. In *2018 15th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, pages 1186–1191, 2018.

[25] Qiuyang Tao, Junkai Wang, Zheyuan Xu, Tony X. Lin, Ye Yuan, and Fumin Zhang. Swing-reducing flight control system for an underactuated indoor miniature autonomous blimp. *IEEE/ASME Transactions on Mechatronics*, 26(4):1895–1904, 2021.

[26] Vito Trianni, Roderich Groß, Thomas H. Labella, Erol Şahin, and Marco Dorigo. Evolving aggregation behaviors in a swarm of robots. In *Advances in Artificial Life*, pages 865–874, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[27] Vanoss, C and Panfilov, Alexander and Hogeweg, P and Siegert, F and Weijer, CJ. Spatial pattern formation during aggregation of the slime mould Dictyostelium discoideum. *JOURNAL OF THEORETICAL BIOLOGY*, 181(3):203–213, 1996.